THEORY OF COMPUTATION MINI PROJECT · PROJECT 15

TURING MACHINE SIMULATOR

Presented by :

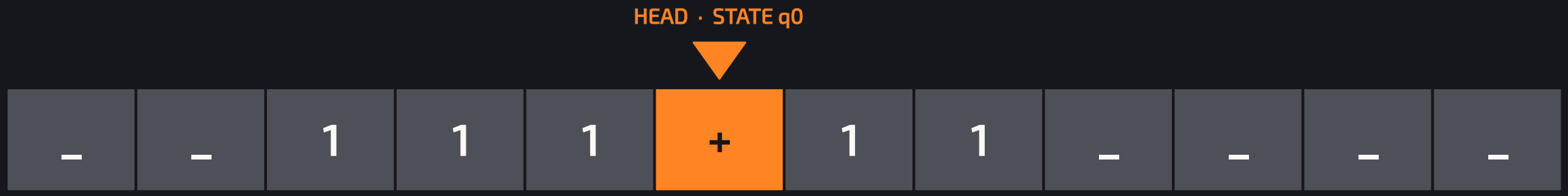
Murapaka Karthik - 4NI24IS110
Pannaga Jois N R - 4NI24IS124
Pannaga S V - 4NI24IS125
Jameer M M - 4NI25IS417

Python · Halting Problem · LLM Safety

2 MACHINES DESIGNED

1 PARADOX PROVED

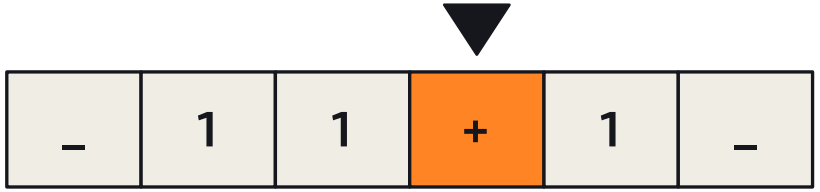
1 SAFETY LESSON



ANYTHING COMPUTABLE IS COMPUTABLE BY A TM.

The Church–Turing thesis: a TM captures what we mean by an algorithm.

A TM IS A TAPE, A HEAD AND A FINITE RULE BOOK



Each step: read the cell, write a symbol, move left or right, change state.

Symbol	Meaning
Q	Finite set of states
Σ	Input alphabet (does not contain the blank)
Γ	Tape alphabet: Σ plus the blank symbol _
δ	Transition function: Q × Γ → Q × Γ × {L, R}
q0	Start state
qaccept / qreject	Halting states: accept and reject

M = (Q, Σ, Γ, δ, q0, qaccept, qreject)

THE PROJECT COVERS FOUR LINKED PARTS

01

FORMAL DESIGN

Unary addition and palindrome checker, designed and traced on paper.

Member 1

02

SIMULATOR

A Python engine that runs any TM given as a transition table, with tape animation.

Member 2

03

HALTING DEMO

The halting paradox demonstrated by construction in code.

Member 3

04

LLM SAFETY

Why undecidability limits what any verifier can guarantee about an AI system.

Member 4

This deck presents the paper designs (part 1), verified against a simulator, and the LLM safety connection (part 4).

TM 1 ADDS BY TURNING + INTO 1, THEN ERASING ONE 1

01

Scan

q0 moves right over the first block of 1s until it finds the + sign.

02

Join

q0 writes 1 over the +, so the two blocks merge. The machine enters q1.

03

Trim

q1 runs to the blank at the end, steps back, and q2 erases the one extra 1.

04

Rewind

q3 moves left to the start of the tape, then accepts.

FORMAL DEFINITION

Q = { q0, q1, q2, q3, qacc, qrej }

Σ = { 1, + } Γ = { 1, +, _ }

start = q0 accept = qacc reject = qrej

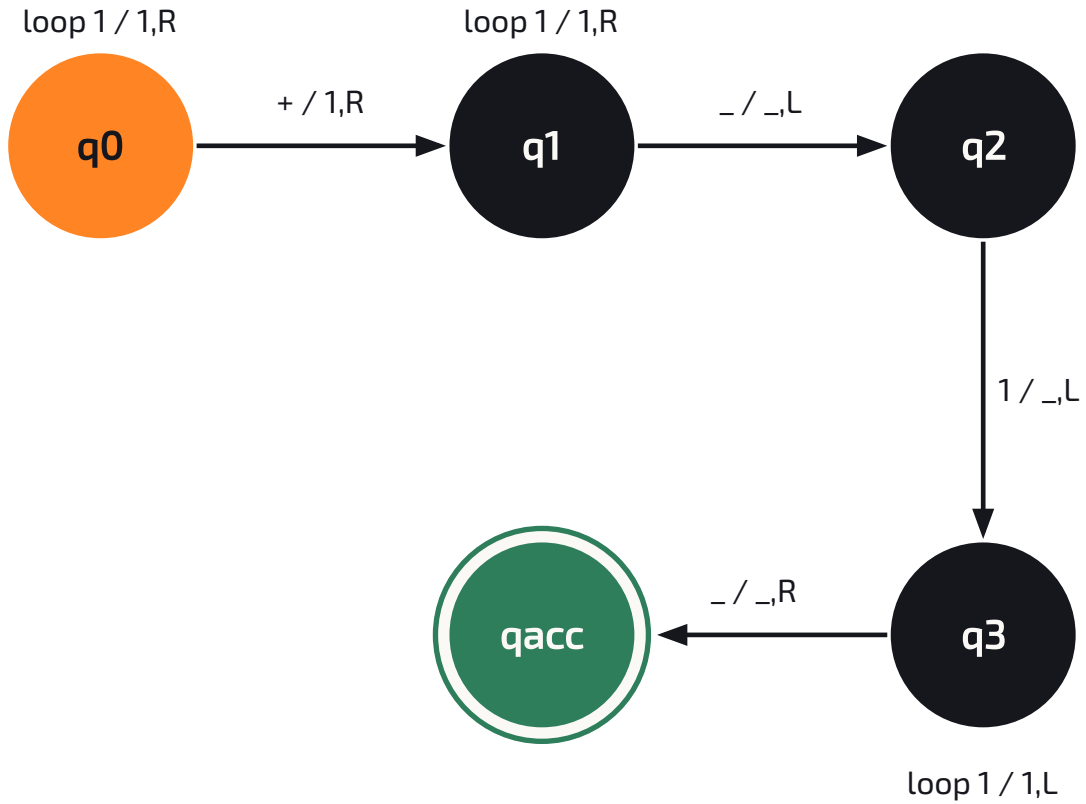
Input 1^m + 1^n → Output 1^(m+n) on the tape, then ACCEPT

TM 1 TRANSITION TABLE AND STATE DIAGRAM

State	reads 1	reads +	reads _
q0	q0, 1, R	q1, 1, R	qrej, _, R
q1	q1, 1, R	qrej, +, R	q2, _, L
q2	q3, _, L	qrej, +, L	–
q3	q3, 1, L	–	qacc, _, R

Each cell is next state, symbol written, head move. A dash means no rule: the machine rejects.

10 rules in total.



label = read / write, move. Any other symbol leads to qrej.

TM 1 TURNS 111+11 INTO 11111 AND ACCEPTS

Step	State	Tape, head in [ ]	What happens
0	**q0**	[1]11+11_	Start, head on the first 1
3	**q0**	111[+]11_	Head reaches the + sign
4	**q1**	1111[1]1_	+ became 1, now in q1
6	**q1**	111111[_]	Blank found at the end
7	**q2**	11111[1]_	Stepped back onto the last 1
8	**q3**	1111[1]__	Last 1 erased, head moves left
12	**q3**	[1]1111__	Head back at the left end
14	**qacc**	[1]1111__	q3 sees the blank: ACCEPT

14

steps to accept 111+11, which is 3 + 2 = 5 in unary.

Checked by simulation: every sum from 0+0 to 7+7 is correct, and malformed inputs are rejected.

Selected steps shown. Unlisted steps repeat the move of the row above.

TM 2 MATCHES THE FIRST AND LAST SYMBOLS, ERASING AS IT GOES

01

Mark

q0 reads the leftmost symbol x, erases it, and remembers x by entering state Rx.

02

Seek

Rx runs right to the blank, then steps back onto the last symbol.

03

Compare

Cx erases the last symbol if it equals x. Any other symbol means REJECT.

04

Return

L runs left to the blank, steps right, and q0 repeats. An empty tape means ACCEPT.

FORMAL DEFINITION

$Q = \{ q0, Ra, Rb, Rc, Rd, Ca, Cb, Cc, Cd, L, qacc, qrej \}$ (12 states)

$\Sigma = \{ a, b, c, d \}$ $\Gamma = \Sigma \cup \{ _ \}$

start = q0 accept = qacc reject = qrej

Rx and Cx exist once per symbol x: the finite control is the machine's memory.

TM 2 TRANSITION RULES FOR ANY SYMBOLS x AND y

State	Reads	Next state	Writes	Moves
q0	x	Rx	_	R
q0	_	qacc	_	R
Rx	any y	Rx	y	R
Rx	_	Cx	_	L
Cx	x	L	_	L
Cx	y ≠ x	qrej	y	R
Cx	_	qacc	_	R
L	any y	L	y	L
L	_	q0	_	R

Rules for Cx with y ≠ x stand for three rules each. Missing rules are rejects.

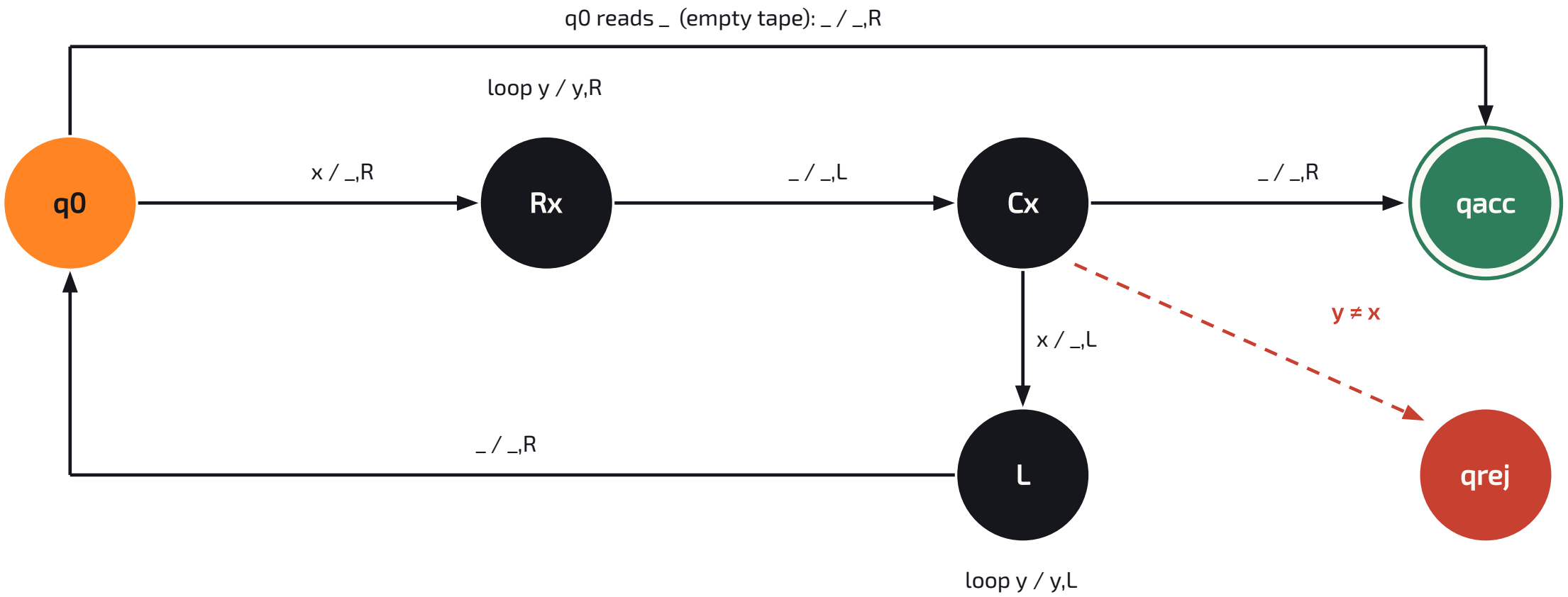
50

concrete transitions once x and y are expanded over the four symbols.

RUNNING TIME

Quadratic. For a^n the machine takes (n+1)(n+2)/2 steps: 45 steps when n = 8.

TM 2 STATE DIAGRAM



label = read / write, move. Rx and Cx are repeated for each x in { a, b, c, d }.

TM 2 ACCEPTS abcba AND REJECTS abcda

abcba · ACCEPT · 21 steps

Step	State	Tape, head in [ ]
0	**q0**	[a]bcba_
5	**Ra**	_bcba[_]
6	**Ca**	_bcb[a]_
10	**L**	[_]bcb__
11	**q0**	_[b]cb__
15	**Cb**	__c[b]__
18	**q0**	__[c]___
20	**Cc**	__[_]___
21	**qacc**	___[_]__

abcda · REJECT · 16 steps

Step	State	Tape, head in [ ]
0	**q0**	[a]bcda_
5	**Ra**	_bcda[_]
6	**Ca**	_bcd[a]_
11	**q0**	_[b]cd__
14	**Rb**	__cd[_]_
15	**Cb**	__c[d]__
16	**qrej**	__cd[_]_

MISMATCH

b vs d

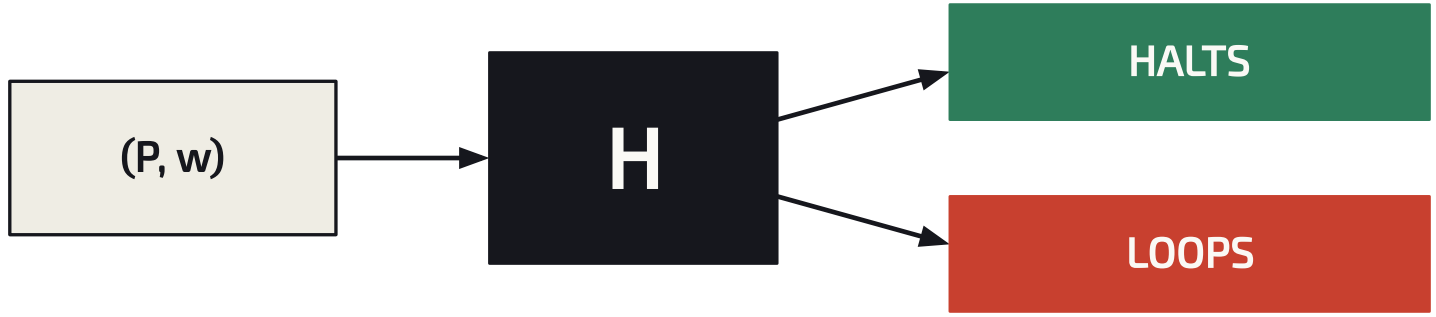
Positions 2 and 4 of abcda. The outer a's matched and were erased first.

Exhaustively checked against a direct palindrome test on all 87,381 strings of length 0 to 8.

THE HALTING PROBLEM IS UNDECIDABLE

Input: a program P and an input w.

Question: does P eventually halt on w, or run forever?



A decider H would have to answer correctly, and always finish, for every possible (P, w). The next slide shows none exists.

Halting is semi-decidable: running P confirms that it halts, but waiting can never confirm that it loops.

NO PROGRAM CAN DECIDE HALTING FOR ALL PROGRAMS.

Turing, 1936. The first problem proved to be undecidable.

ASSUME A HALTING DECIDER EXISTS, THEN BUILD A PROGRAM THAT BREAKS IT

1

ASSUME

H(P, w) always halts and answers correctly: halts or loops.

2

BUILD D

D(P) runs H(P, P). If H says halts, D loops forever. If H says loops, D halts.

3

SELF-APPLY

Run D on its own code: D(D). H must give an answer about D(D).

4

CONTRADICT

If H says D(D) halts, D loops. If H says loops, D halts. H is wrong both ways.

Conclusion: the assumption was false, so no such H can exist.

This is the same self-referential trick Member 3's demonstration code builds in Python.

NO GENERAL CHECKER FOR BEHAVIOR.

Rice's theorem, 1953: halting is just one case.

RICE'S THEOREM: ANY NON-TRIVIAL QUESTION ABOUT BEHAVIOR IS UNDECIDABLE

Question about a program	Decidable in general?
Does it halt on input w?	**No**
Does it ever print the word X?	**No**
Is it equivalent to program Q?	**No**
Does its source text contain the word print?	**Yes: a text property**
Does it halt within N steps?	**Yes: a bounded run**

Any property of what a program does (not how its code looks) that some programs have and others lack cannot be decided by one algorithm for all programs.

WHY UNDECIDABILITY MATTERS FOR LLM SAFETY

Halting problem	We cannot always predict whether an AI agent that loops over tool calls will ever stop.
Rice's theorem	No general checker can decide "will this system ever produce harmful output?" for every system and input.
Self-reference (D on D)	A verifier asked to judge a system that contains itself meets the same paradox, so a model cannot fully certify its own output.

Bottom line: no system can perfectly predict what an arbitrary LLM or agent will output.

UNDECIDABILITY LIMITS GUARANTEES, IT DOES NOT MAKE SAFETY HOPELESS

IT SAYS

- No single universal verifier is correct for every program and every behavior.
- A perfect, general proof that an open-ended AI system is never harmful is out of reach.
- Agents with tools, loops and external memory behave like general computation.

IT DOES NOT SAY

- That specific systems cannot be tested, or specific narrow properties checked.
- That any particular model is unsafe.
- That a bare LLM with a fixed context window is strictly undecidable: it is finite-state, but with a state space far too large to enumerate.

Stating this limit precisely keeps the argument honest.

PRACTICAL SAFETY WORKS AROUND THE LIMIT

BOUNDED CHECKS

Timeouts, step limits and token limits. "Halts within N steps" is decidable even though halting is not.

NARROW PROPERTIES

Verify specific restricted properties, such as sandboxed tool permissions, not "all good behavior".

LAYERED DEFENSES

Testing, red-teaming, monitoring and human oversight together, since no single check is complete.

RUNTIME MONITORING

We cannot always predict an output, but we can run the system and observe what it actually does.

NO PERFECT PREDICTOR.

Safe AI needs layered, bounded and empirical checks.

WHAT THIS PROJECT SHOWS

1. **A TM is a precise model of computation.**
   Both machines were designed on paper and verified by simulation.

2. **Halting is undecidable.**
   Proved by assuming a decider and building D, which defeats it.

3. **Rice's theorem generalizes it.**
   Questions about program behavior have no general decider.

4. **LLM safety inherits the limit.**
   No system can perfectly predict an arbitrary LLM's output, so safety relies on layers.